# 计算机组成原理

## 中央处理器

# 考试大纲

## 五、中央处理器（CPU）

（一）CPU的功能和基本结构

（二）指令执行过程

（三）数据通路的功能和基本结构

（四）控制器的功能和工作原理

（五）异常和中断机制

1.异常和中断的基本概念

2.异常和中断的分类

3.异常和中断的检测与响应

（六）指令流水线

1.指令流水线的基本概念

2.指令流水线的基本实现

3.结构冒险、数据冒险和控制冒险的处理

4.超标量和动态流水线的基本概念

（七）多处理器基本概念

1.SISD、SIMD、MIMD、向量处理器的基本概念

2.硬件多线程的基本概念

3.多核处理器（multi-core）的基本概念

4.共享内存多处理器 （SMP）的基本概念

# CPU的基本功能

中央处理器（CPU）由运算器和控制器组成，其核心功能是解释计算机指令以及处理计算机软件中的数据。

### 1. 指令控制 (程序控制)

控制程序严格按照规定的顺序执行，这是CPU的首要任务。决定指令执行顺序的是程序计数器(PC)。

### 2. 操作控制

一条指令由若干操作信号组成。CPU管理并产生由内存取出的每条指令的操作信号，送往不同的部件以控制对应动作。

### 3. 时间控制

对各种操作实施时间上的定时。指令的执行过程受到严格的时钟信号同步控制。

### 4. 数据加工

对数据进行算术运算和逻辑运算处理。这是CPU的最根本的任务，主要由运算器(ALU)完成。

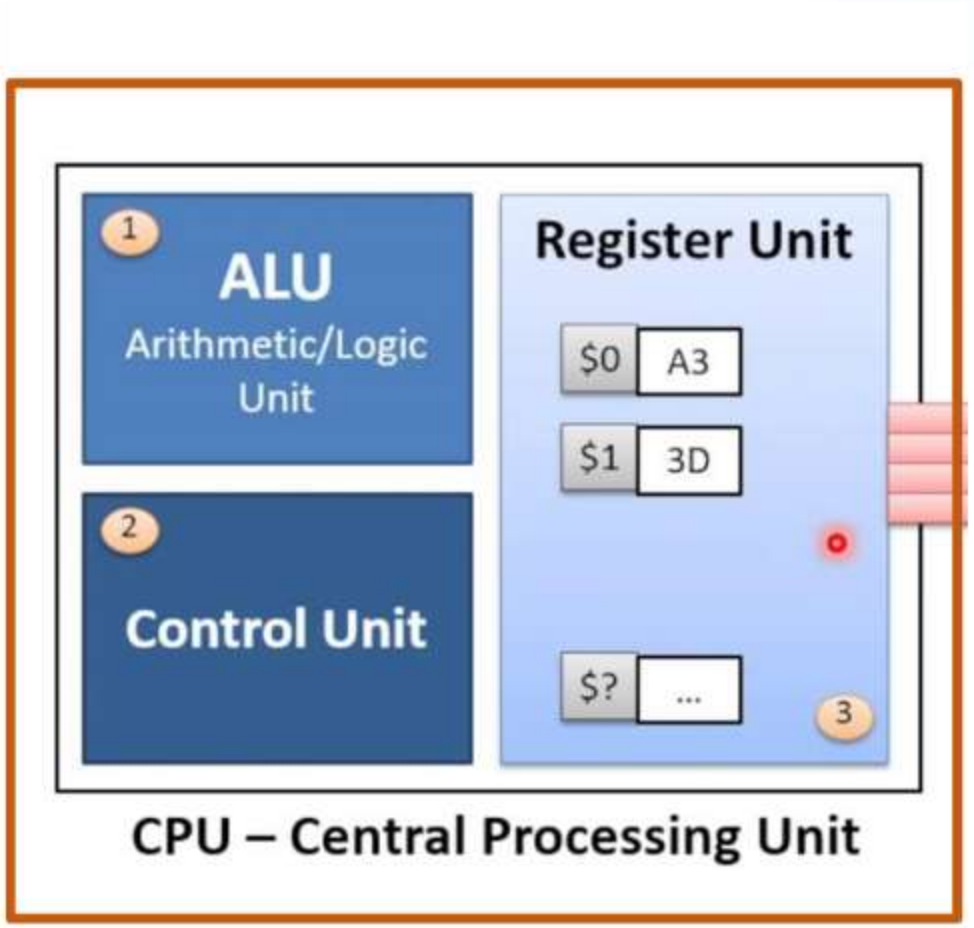
# CPU基本结构宏观图

## 逻辑部件划分

为了实现前述功能，CPU在逻辑上可分为两大部分：

- **运算器 (Datapath / ALU)：** 负责数据的加工处理。是计算机对信息进行处理和运算的部件。
- **控制器 (Control Unit)：** 整个系统的“指挥中枢”。负责协调和控制计算机各部件有条不紊地工作。

**总线 (Bus)：** CPU内部部件之间通过内部总线相连，实现数据的高速传输。而CPU与主存、I/O之间通过系统总线（数据线、地址线、控制线）交互。



# 运算器与控制器详解

## 运算器 (ALU核心)

**功能：** 接收控制器的命令，进行算术运算（加减乘除）和逻辑运算（与或非移位）。

**核心部件：**

- 算术逻辑单元 (ALU)：真正的执行部件。
- 暂存寄存器：用于缓存由主存读取的数据，避免破坏原有数据。
- 累加寄存器 (ACC)：最常见的通用寄存器。
- 通用寄存器组 (GPRs)：如AX, BX, CX, DX，存放操作数及各种地址信息。
- 程序状态字寄存器 (PSW)：存放运算标志（如溢出OF、进位CF、零标志ZF等）。

## 控制器 (CU核心)

**功能：** 从主存取指令、分析指令并产生相应的操作控制信号。

**核心部件：**

- 程序计数器 (PC)：存放下一条要执行指令的地址。
- 指令寄存器 (IR)：存放当前正在执行的指令
- 指令译码器 (ID)：对操作码字段进行分析译码。
- 微操作信号发生器：根据IR的内容和时序信号，产生控制整个计算机系统所需的各种控制信号。
- 存储器数据/地址寄存器 (MDR/MAR)：用于与主存交互信息的缓冲。

# 寄存器的“透明性”

在汇编语言程序员眼中，CPU内的寄存器分为两类。

## 用户可见寄存器

汇编程序员**可以**通过指令直接访问、修改其内容的寄存器。

- 通用寄存器 (如 R0~R31, EAX, EBX)
- 程序状态字寄存器 (PSW)
- 程序计数器 (PC) (通过转移指令修改)
- 基址寄存器、变址寄存器、段寄存器

## 用户不可见 (透明) 寄存器

由CPU硬件自动控制使用，汇编程序员**无法**直接操作的寄存器。

- 指令寄存器 (IR)
- 主存地址寄存器 (MAR)
- 主存数据寄存器 (MDR)
- 暂存寄存器 (用于ALU输入端)
- 微程序相关的寄存器 (如微地址寄存器 CMAR)

# 运算器中的主要寄存器特写



## 累加器 (ACC)

Accumulator。这是一个通用寄存器，通常用于存放操作数或运算结果。在单地址指令系统中，ACC默认提供另一个操作数，并接收运算结果。是最繁忙的寄存器之一。



## 程序状态字 (PSW)

存放由算术/逻辑运算指令建立的各种状态标志信息（如进位CF、溢出OF、零ZF、符号SF等），以及控制信息（如中断允许标志IF、方向标志DF）。是分支跳转指令的重要依据。



## 暂存寄存器

对于双操作数指令（如加法），一个操作数来自主存，一个来自ACC。为了防止从主存读出的数在运算过程中破坏内部总线上的其他信息，需将其暂存于此。对程序员完全透明。

# 控制器中的主要寄存器特写



## 程序计数器 (PC)

存放**下一条**欲执行指令的主存地址。具有自增功能（通常PC = PC + 1/指令字长）。分支转移指令通过改变PC的值来实现程序跳转。是用户可见的。



## 指令寄存器 (IR)

存放**当前正在执行**的指令。其操作码部分送至指令译码器，地址码部分送至地址形成部件产生有效地址。对程序员透明。



## MAR & MDR

**MAR:** 主存地址寄存器，存放欲访问的存储单元地址。

**MDR:** 主存数据寄存器，存放向主存写入或从主存读出的数据。两者是CPU与主存交互的桥梁，均对程序员透明。

# 指令周期、机器周期与时钟周期

## 基本概念层次

- **指令周期 (Instruction Cycle):** CPU从主存取出并执行完一条指令所需的全部时间。包含若干个机器周期。
- **机器周期 (Machine Cycle) / CPU周期:** 也称CPU周期。通常把一个指令周期划分为若干个阶段，每个阶段完成一项基本操作（如取指、访存）。
- **时钟周期 (Clock Cycle) / 节拍 (T状态):** CPU操作的**最基本单位**，是主频的倒数。一个机器周期包含若干个时钟周期。

## 指令周期的组成

一个完整的指令周期通常包含以下几个工作周期（标志触发器控制状态转换）：

- **取指周期 (FE):** 必有。取指令。
- **间址周期 (IND):** 可选。取有效地址。
- **执行周期 (EX):** 必有。执行指令。
- **中断周期 (INT):** 可选。响应中断请求。

**指令周期 = 取指周期 + (间址周期) + 执行周期 + (中断周期)**

# 取指周期 (Fetch Cycle) 及微操作

**任务：** 根据PC中的内容从主存提取指令送到IR中，并使PC指向下一条指令。

## 微操作序列:

1. **PC → MAR** : 将当前指令地址送主存地址寄存器
2. **1 → R** : 向主存发送读命令 (Read)
3. **M(MAR) → MDR** : 将主存指定单元内容读入MDR
4. **MDR → IR** : 将MDR中的指令送入指令寄存器
5. **(PC) + 1 → PC** : 形成下一条指令地址 (假设指令单字长)



**Memory → CPU**

# 间址周期 (Indirect Cycle) 及微操作



**Address → Real Address**

**任务：** 针对间接寻址的指令，取操作数的有效地址。将指令中的形式地址作为指针，去主存中取真正的操作数地址。

## 微操作序列:

1. **Ad(IR) → MAR** : 将指令中的地址码(形式地址)送入MAR
2. **1 → R** : 发出读操作命令
3. **M(MAR) → MDR** : 将操作数的有效地址读入MDR
4. (可选) **MDR → Ad(IR)** : 将有效地址送回IR的地址段，以备执行周期使用

# 执行周期 (Execution Cycle)

**任务：** 根据IR中的指令操作码和操作数（或地址），通过ALU执行相应的运算操作。

执行周期的微操作因指令而异，没有统一的微操作序列。常见类型：

## 非访存指令 (如 CLA)

清空累加器 (Clear ACC)。

微操作: 0 → ACC

## 访存指令 (如 ADD X)

将主存X单元的内容与ACC相加。

1. Ad(IR) → MAR
2. 1 → R, M(MAR) → MDR
3. (ACC) + (MDR) → ACC

# 中断周期 (Interrupt Cycle) 及微操作

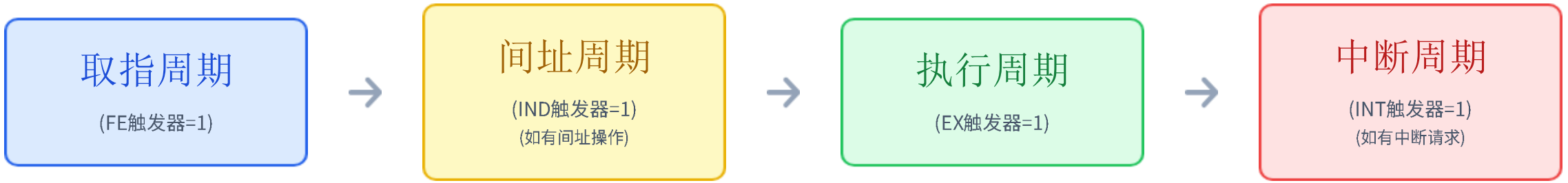
**任务：** 在一条指令执行完毕后，CPU检查是否有未屏蔽的中断请求。如果有，则进入中断周期，主要任务是**保存断点**并形成中断服务程序的入口地址。

## 微操作序列 (硬件完成):

1. **CU控制SP减1, SP → MAR** : 准备压栈，将栈顶地址送MAR
2. **1 → W** : 发出写操作命令
3. **PC → MDR** : 将PC（断点）送入MDR
4. **MDR → M(MAR)** : 将断点存入主存栈中
5. **向量地址 → PC** : 将中断服务程序入口地址送入PC，准备执行中断处理程序
6. **0 → EINT** : 关中断（硬件自动完成）

> ⚠注意
>
> 中断周期由硬件直接实现（硬件隐指令），它不仅保存了PC（断点），为了不被新的中断打断，必须立即硬件关中断（修改PSW中的IF位为0）。

# 指令执行完整流程图




每个周期结束时，根据条件（如是否间址、是否有中断）设置相应的状态触发器，决定下一个周期进入哪个阶段。

# 数据通路的定义与功能

> "数据通路是指数据在功能部件之间传送的路径。其功能是实现**CPU**内部的运算器与寄存器及寄存器之间的数据交换。"

## 数据通路的基本组成：

- 组合逻辑元件：如ALU、多路选择器(MUX)、三态门。起数据加工和路径选择作用。
- 时序逻辑元件：如通用寄存器、状态寄存器。起数据保存作用。
- 控制信号：由CU发出，控制三态门的开闭、MUX的选择端等，决定数据流向。

## 建立数据通路的核心原则：

在同一个时钟周期内，连接在**同一条总线**上的所有部件，只能有一个部件向总线发送数据（输出使能有效），但可以有多个部件同时从总线接收数据（输入使能有效）。

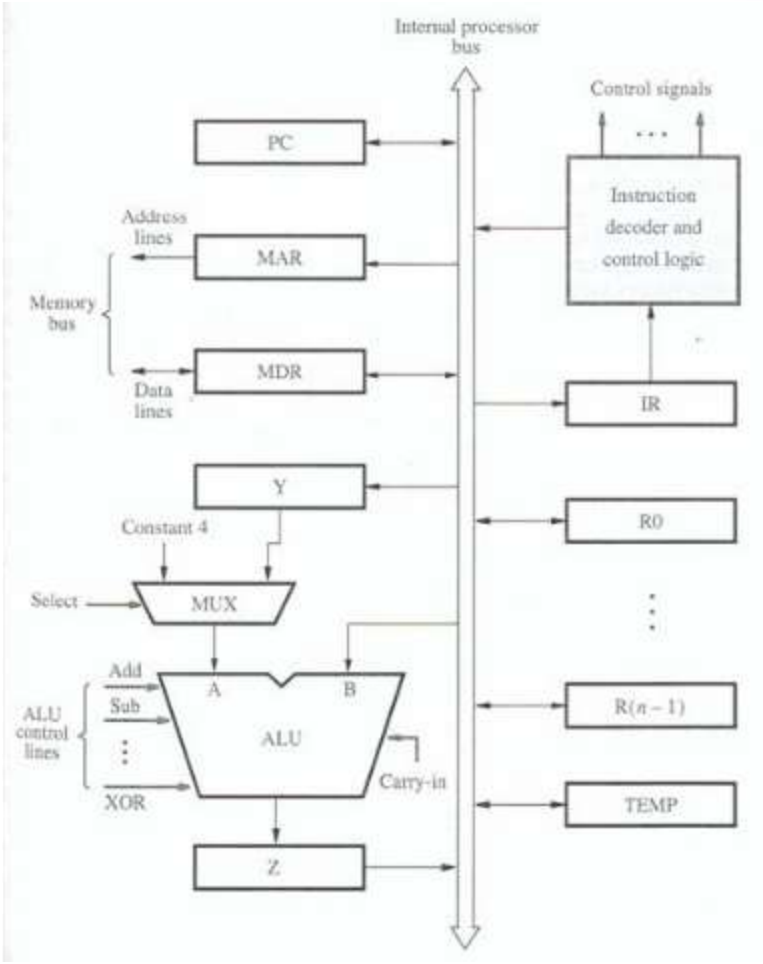
# CPU内部单总线结构

## 单总线结构特点

所有寄存器的输入端和输出端都连接到一条公共总线上。

- **优点：** 结构简单，控制电路少，连线成本低。
- **缺点：** 存在严重的资源冲突（总线竞争）。在同一个时钟周期内只允许发生一次数据传送。

**注意：** 因为ALU需要两个输入端，而在单总线上同一时刻只能提供一个数据。因此，ALU的一个输入端必须接一个**暂存寄存器 (Y)**，以缓存第一个操作数。



# 多总线结构 vs 专用数据通路

## 内部三总线结构

CPU内部设置多条公共总线（如两条源数据总线，一条结果总线）。

**ALU连接方式：** ALU的两个输入端分别直接连到两条总线上，输出端连到第三条总线上。

**优点：** 可以在一个时钟周期内同时准备好两个操作数并写回结果，大大提高了数据传送和处理的效率。

## 专用数据通路结构

根据指令执行过程中的数据和地址的流动方向，在相关部件之间建立专属的物理连接（硬件连线）。不再共享总线。

通常使用**多路选择器 (MUX)** 或三态门来控制数据的输入源。

**优点：** 并发度极高，性能最好。
**缺点：** 硬件连线极其复杂，成本高。

# 数据传送 1: 寄存器之间

### 寄存器之间的数据传送

例如：把寄存器PC的内容传送给寄存器MAR，实现微操作 PC → MAR。

#### 基于单总线结构

1. **PCout 有效：** 允许PC的内容输出到内部总线Bus。
2. **MARin 有效：** 允许 MAR 从内部总线 Bus 接收数据。

(PC) → Bus → MAR

#### 基于专用数据通路

由于PC和MAR之间有直接连线：

1. 直接使得 **C0 有效** (假设C0是控制该直连通道三态门的控制信号)。

(PC) → MAR

# 数据传送 2: 主存与CPU之间

主存与CPU之间的数据交互必须经过 MAR (地址) 和 MDR (数据)。
以 CPU 从主存读取指令 (取指周期核心部分) 为例：

## 读主存微操作序列及控制信号 (基于内部单总线)

| 微操作 | 有效控制信号 | 含义 |
| --- | --- | --- |
| (PC) → MAR | PCout, MARin | 将PC的内容放到系统地址线上 |
| 1 → R | MemR (读控制线) | 向主存发读命令 |
| M(MAR) → MDR | MDRinE | 主存数据经外部数据总线送入MDR (E代表External) |

# 数据传送 3: 执行算术或逻辑运算

以单总线结构下执行加法指令 ADD R0, R1 (R0 = R0 + R1) 为例：

```
T1: (R0) → Bus → Y          /* R0out, Yin 有效。将源操作数缓存至Y */
T2: (R1) → Bus → ALU端      /* R1out 有效。R1内容上总线 */
    (Y) + (R1) → Z          /* ALUadd, Zin 有效。ALU执行加法，结果存入暂存寄存器Z */
T3: (Z) → Bus → R0          /* Zout, R0in 有效。结果写回R0 */
```

💡**解析：** 单总线结构ALU的输入和输出不能在同一个时钟周期进行，否则总线会冲突。因此必须引入 Y（输入暂存）和 Z（输出暂存）。

# 硬布线控制器 (Hardwired Control)

## 设计思想

将微操作控制信号视为一个复杂的逻辑函数。

其输入包括：

- 指令操作码译码输出 (IR)
- 状态标志 (PSW条件码)
- 节拍电位和工作周期信号 (时序发生器)

通过布尔代数化简，最后用门电路（与非门、触发器等）物理连线实现。

## 特点与应用

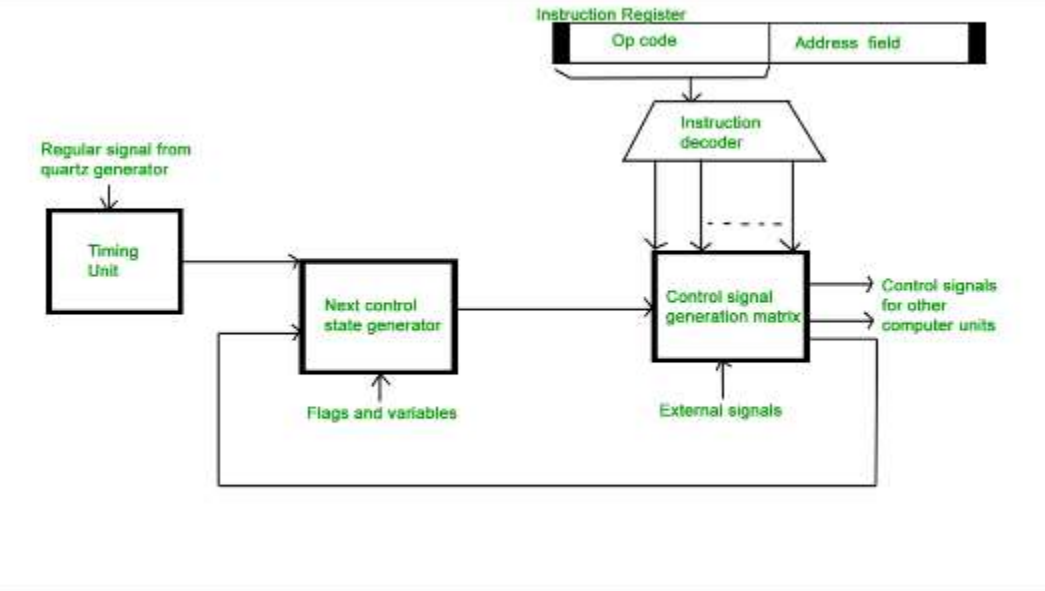
- **优点：** 纯硬件执行，由于是组合逻辑直接输出，没有读取微指令的时间，因此**速度极快**。
- **缺点：** 结构庞大复杂；一旦设计完成，要想增加新指令或修改指令极其困难，需要重新设计整个物理电路。
- **应用场景：** 主要应用于RISC（精简指令集计算机）架构中，因为RISC指令种类少，格式规整。

# 微程序控制器 (Microprogrammed Control)

## 核心概念

采用**存储逻辑**实现。把微操作信号代码化（微指令），存放在一个只读存储器（Control Memory, CM）中。

- **微命令与微操作：** 微命令是CU发出的控制信号，微操作是执行部件的操作。
- **微指令：** 在一个节拍内发出的一组微命令，外加下一条微指令的地址。
- **微程序：** 若干条微指令构成的序列。**一条机器指令对应一个微程序。**
- **控制存储器 (CM)：** 位于CPU内部，用ROM实现，用于存放所有微程序。



# 微指令格式 (水平型 vs 垂直型)

## 水平型微指令

一条微指令能同时定义并执行多个并行的微命令。

操作控制字段 | 下址字段

- **直接编码：** 每一位代表一个微命令（1有效）。位长很长。
- **字段直接编码：** 互斥的微命令分在一组并译码。例如3位可表示7个互斥微命令(000不用)。
- **优点：** 微程序短，执行速度快，并行度高。
- **缺点：** 微指令字太长，CM容量大。

## 垂直型微指令

类似于机器指令的操作码方式，一条微指令通常只能完成一个（或几个）微操作。

微操作码 | 目的地址 | 源地址

- **特点：** 采用微操作码译码。
- **优点：** 微指令字短，简单，规整。
- **缺点：** 微程序长，执行速度慢，并行度低。

* 注：目前绝大多数计算机采用水平型微指令。

# 微地址的形成方式（下址生成）

执行微指令时，如何确定下一条要执行的微指令的地址？（408常考点）

## 1 断定方式（直接由下址字段提供）

微指令的"下址字段"直接给出下一条微指令的地址。如果遇到分支转移，则通过特定的测试网络（结合状态标志）来修改下址。

## 2 根据机器指令的操作码形成

当机器指令取至IR后，将**操作码经微地址形成部件译码**，直接产生该机器指令对应的微程序入口地址，送入微地址寄存器(CMAR)。这是由取指周期跳转到执行周期的关键一步。

## 3 增量计数器方式

类似于PC，顺序执行时：(CMAR) + 1 → CMAR。这种方式要求微指令必须连续存放。

# 核心总结：硬布线 vs 微程序控制器比较

| 比较项目 | 硬布线控制器 (Hardwired) | 微程序控制器 (Microprogrammed) |
| --- | --- | --- |
| **实现原理** | 纯硬件逻辑门电路 | 存储逻辑 (微指令存入ROM) |
| **执行速度** | **极快** (无访存读取微指令过程) | **较慢** (需要读取CM中的微指令) |
| **灵活性/修改难度** | 差。修改指令系统需重造硬件 | 优。只需修改CM中的微程序即可 |
| **规整性** | 繁琐，不规整 | 规整，类似软件编程 |
| **应用架构** | RISC (精简指令集计算机) | CISC (复杂指令集计算机) |

# 异常与中断的基本概念

## 异常 (Exception / 内中断)

**源头：** 产生于CPU内部，与当前执行的指令**密切相关**。

**原因：** 通常由非法操作码、地址越界、算术溢出、缺页故障等引起。或者程序故意触发（软中断）。

**时机：** 异常可以在指令执行过程中的任何阶段产生并立即被CPU响应。

## 中断 (Interrupt / 外中断)

**源头：** 产生于CPU外部，与当前执行的指令**无关**。

**原因：** 通常由外部外设的请求引起，如键盘输入、定时器到期、网络数据到达等。

**时机：** CPU只有在**一条指令执行完毕后**（即中断周期）才会去检查并响应外中断。

# 异常的分类 (故障、陷阱、终止)



## 故障 (Fault)

由指令执行引起的异常。可能被故障处理程序修复。**修复后，返回到引起故障的指令重新执行。**

**经典例子：缺页异常(Page Fault)。**



## 陷阱 (Trap / 陷入)

是一种有意的、预先安排的“异常”。程序主动执行特定指令触发，用于系统调用。**处理后，返回到陷阱指令的下一条指令执行。**

**经典例子：系统调用(Syscall)、断点。**



## 终止 (Abort)

出现了导致系统无法继续执行的致命硬件错误。处理程序无法修复，直接终止应用程序，甚至系统重启。

**经典例子：控制器出错、内存奇偶校验错。**

# 中断的分类 (可屏蔽 vs 不可屏蔽)

## 可屏蔽中断 (Maskable Interrupt, INTR)

受CPU内部标志位（如PSW中的IF位，Interrupt Flag）控制。如果 IF=0 (关中断)，CPU将忽略此类请求；如果 IF=1 (开中断)，CPU才会响应。

大多数I/O设备的中断请求都属于可屏蔽中断。

## 不可屏蔽中断 (Non-maskable Interrupt, NMI)

不受中断允许标志位 (IF) 的控制。只要产生，CPU必须无条件地在当前指令执行完毕后立即响应。

通常用于处理紧急的硬件故障，如电源掉电(Power failure)。

# 异常和中断的检测机制

## 异常的检测

异常的检测是由CPU在**执行指令的过程中**通过硬件逻辑同步完成的。

- 取指阶段检测：存储器保护错、缺页等。
- 译码阶段检测：非法操作码。
- 执行阶段检测：算术溢出、除数为0等。

**发现异常立即响应，无需等待指令结束。**

## 中断的检测

中断的检测是异步的。外部设备通过中断请求线(INTR/NMI)向CPU发信号。

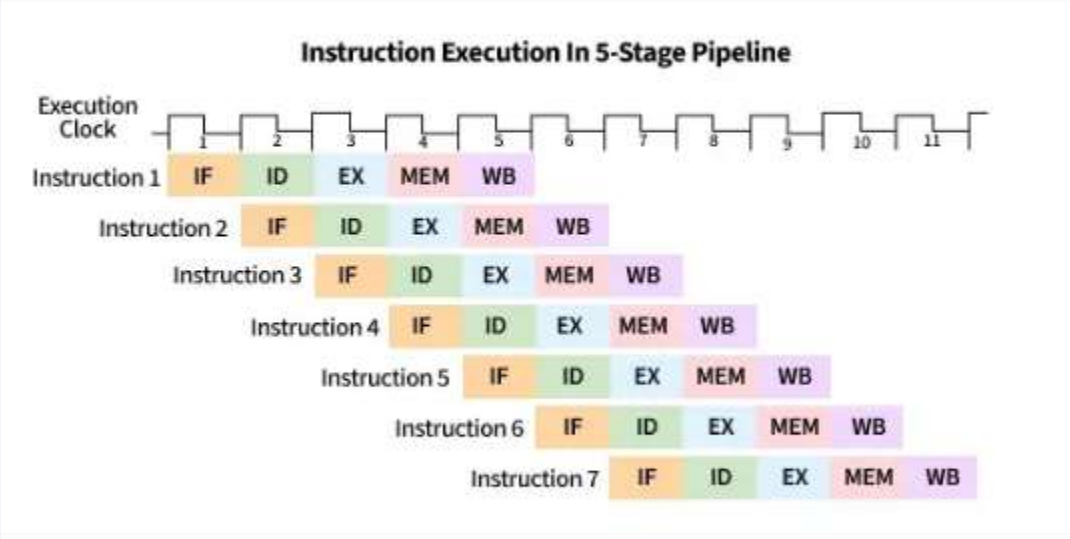
- CPU内部的指令执行控制逻辑在**每条指令执行的最后一个时钟周期**（即结束前），会去采样中断请求线。
- 如果有请求且满足响应条件（如IF=1），则进入中断响应周期。

**必须等待当前指令执行完毕。**

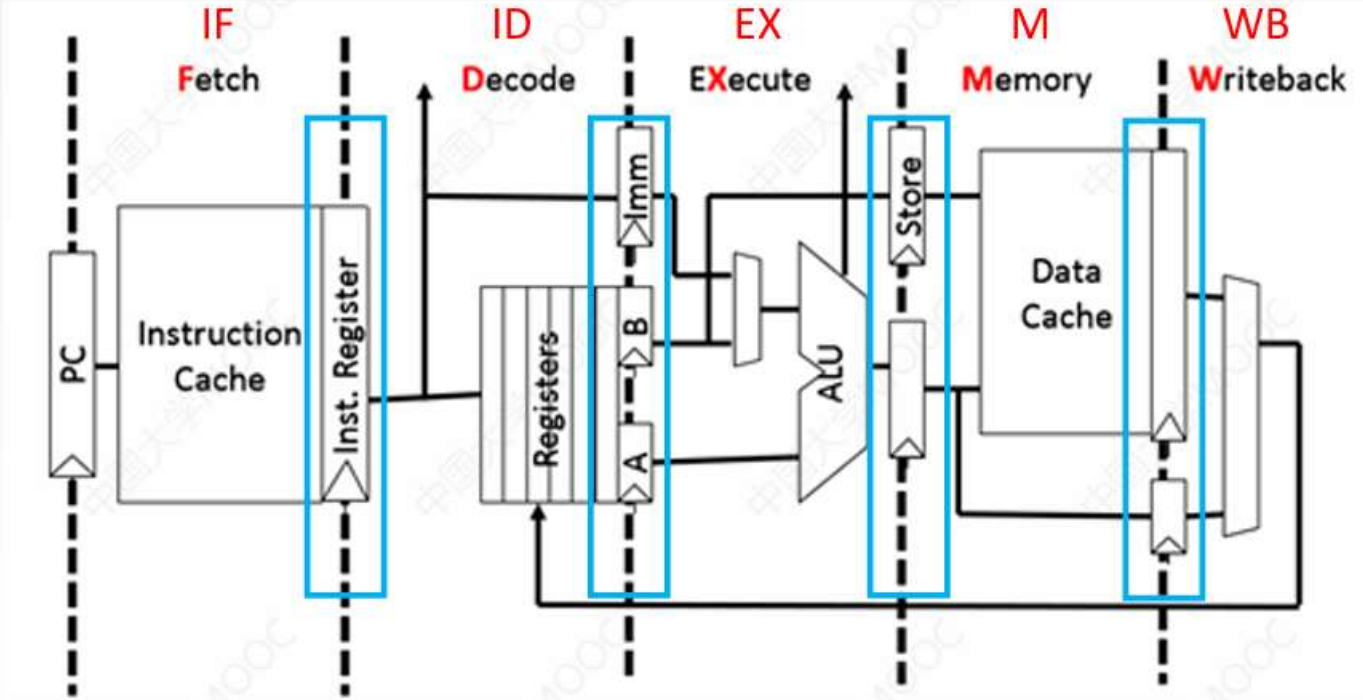
# 五段流水线的基本实现 (MIPS架构经典)

经典的RISC五段指令流水线划分：

- **1. IF (Instruction Fetch) - 取指:** 依据PC从指令Cache取指令。
- **2. ID (Instruction Decode) - 译码/读寄存器:** 识别指令类型，并从通用寄存器堆(Register File)读出源操作数。
- **3. EX (Execute) - 执行/计算地址:** ALU执行算术逻辑运算，或计算访存地址（Load/Store指令），或计算转移目标地址（Branch指令）。
- **4. MEM (Memory Access) - 访存:** Load/Store指令在此阶段读写数据Cache。其他指令此阶段空转。
- **5. WB (Write Back) - 写回:** 将运算结果或读出的数据写回寄存器堆。



# 五段式指令流水线





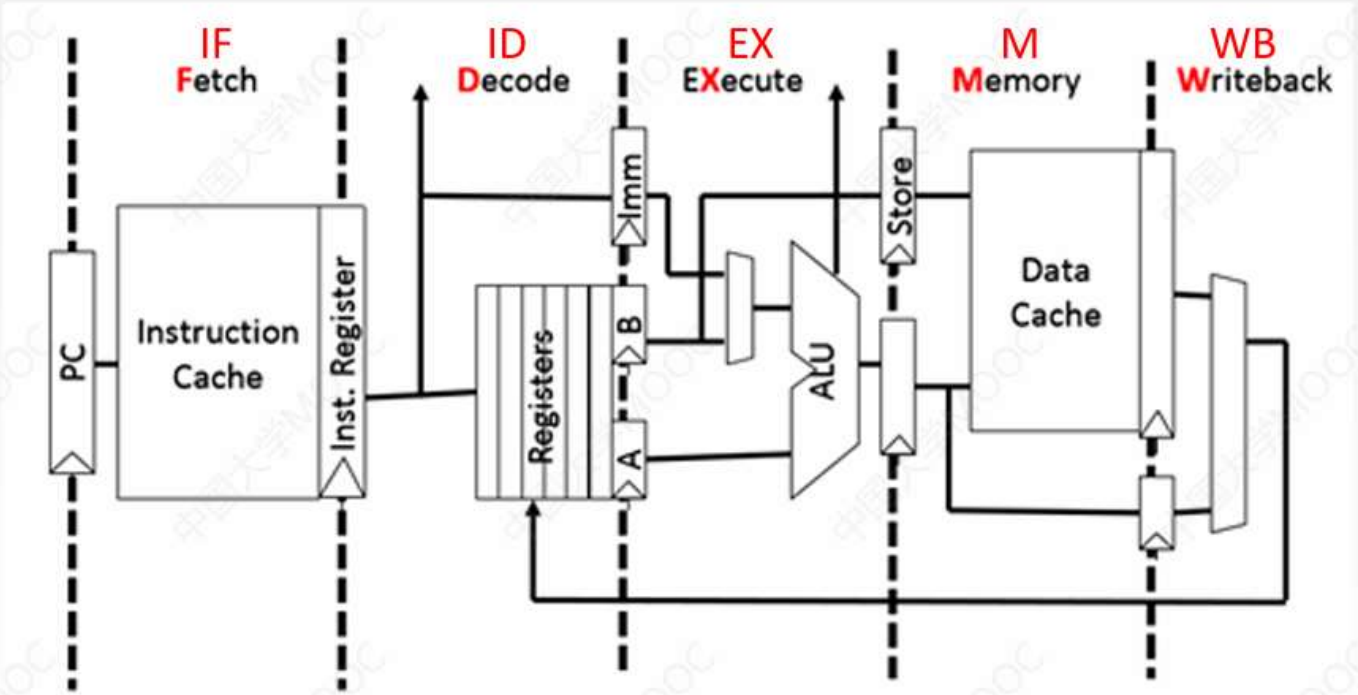


①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ⑤WB写回寄存器

考试中常见的五类指令：

- 运算类指令、LOAD指令、STORE指令、条件转移指令、无条件转移指令

# 运算类指令的执行过程



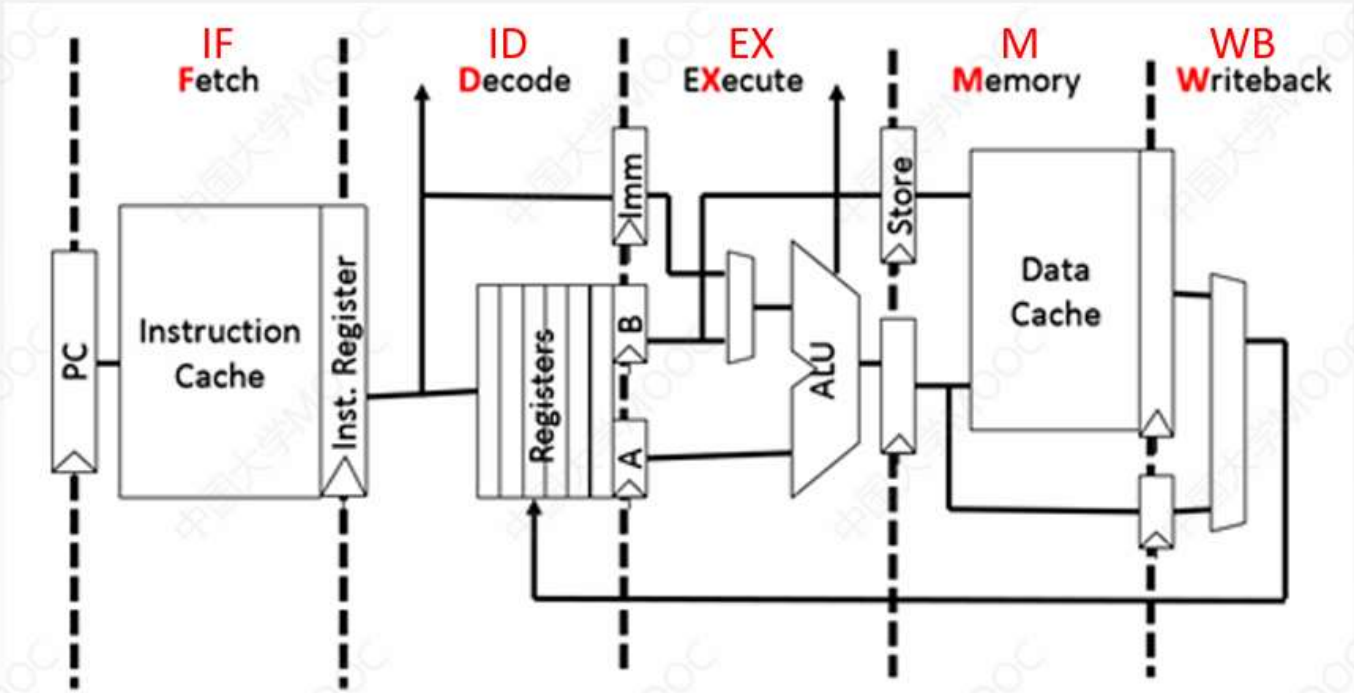

注：
$R_s$ 指源操作数（source）
$R_d$ 指目的操作数（destination）

运算类指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：取出操作数至ID段锁存器
EX：运算，将结果存入EX段锁存器
M：空段
WB：将运算结果写回指定寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行→ ~~④M访存~~ → ⑤WB写回寄存器

| 运算类指令举例 | 指令的汇编格式 | 功能 |
| --- | --- | --- |
| 加法指令（两个寄存器相加）： | ADD Rs,Rd | (Rs)+(Rd)→Rd |
| 加法指令（寄存器与立即数相加）： | ADD #996,Rd | 996+(Rd)→Rd |
| 算数左移指令： | SHL Rd | (Rd)<<<2→Rd |

# LOAD指令的执行过程



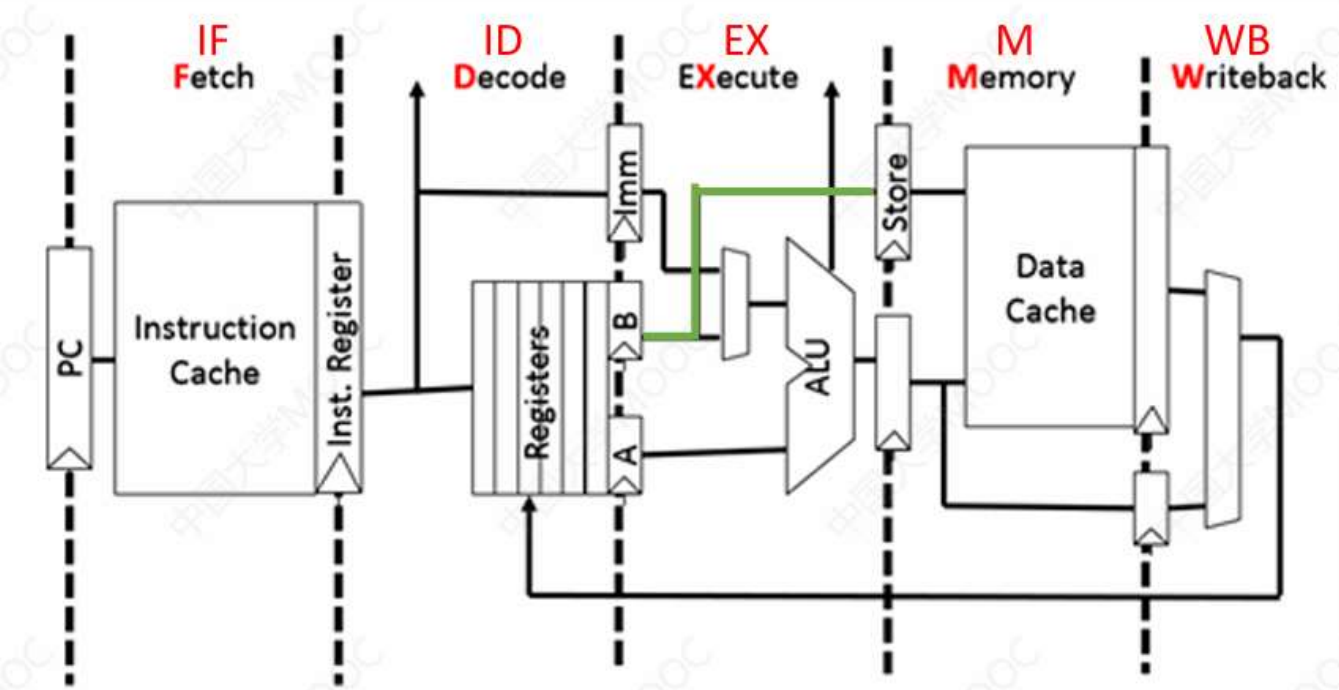

LOAD指令
IF：根据PC从指令Cache取指令至IF段的锁存器
ID：将基址寄存器的值放到锁存器A，将偏移量的值放到Imm
EX：运算，得到有效地址
M：从数据Cache中取数并放入锁存器
WB：将取出的数写回寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ⑤WB写回寄存器

| 指令的汇编格式 | 功能 |
| --- | --- |
| LOAD Rd,996(Rs) | (996+(Rs))→Rd |
| 或简写为：<br>LOAD Rd,mem | (mem)→Rd |

通常，RISC处理器只有“取数LOAD”和“存数STORE”指令才能访问主存

# STORE指令的执行过程




STORE指令

IF：根据PC从指令Cache取指令至IF段的锁存器

ID：将基址寄存器的值放到锁存器A，将偏移量的值放到Imm。将要存的数放到B

EX：运算，得到有效地址。并将锁存器B的内容放到锁存器 Store。

M：写入数据Cache

WB：空段

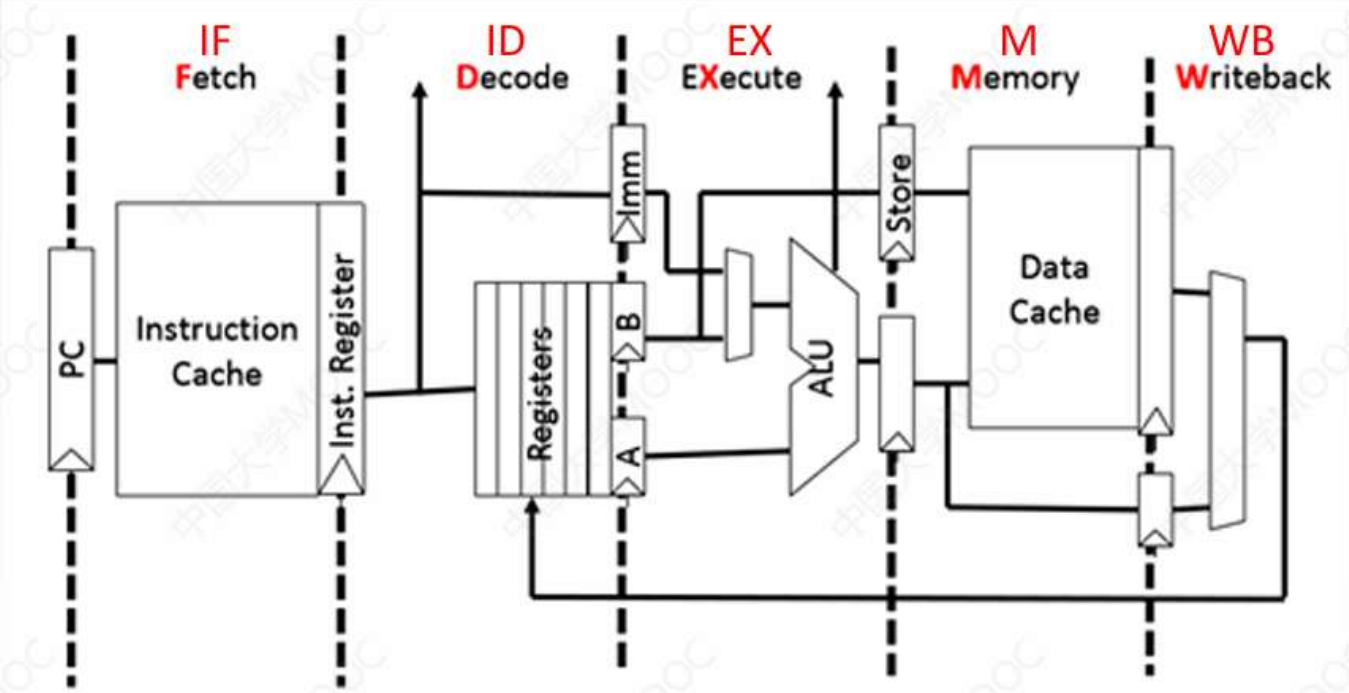
①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ~~⑤WB写回寄存器~~

| 指令的汇编格式 | 功能 |
| --- | --- |
| STORE Rs,996(Rd) | Rs→(996+(Rd)) |

或简写为：

| | |
| --- | --- |
| STORE Rs,mem | Rs→(mem) |

# 条件转移指令的执行过程




转移类指令常采用相对寻址

条件转移指令

IF：根据PC从指令Cache取指令至IF段的锁存器

ID：进行比较的两个数放入锁存器A、B；偏移量放入 Imm

EX：运算，比较两个数

M：将目标PC值写回PC（左图没画全）

WB：空段

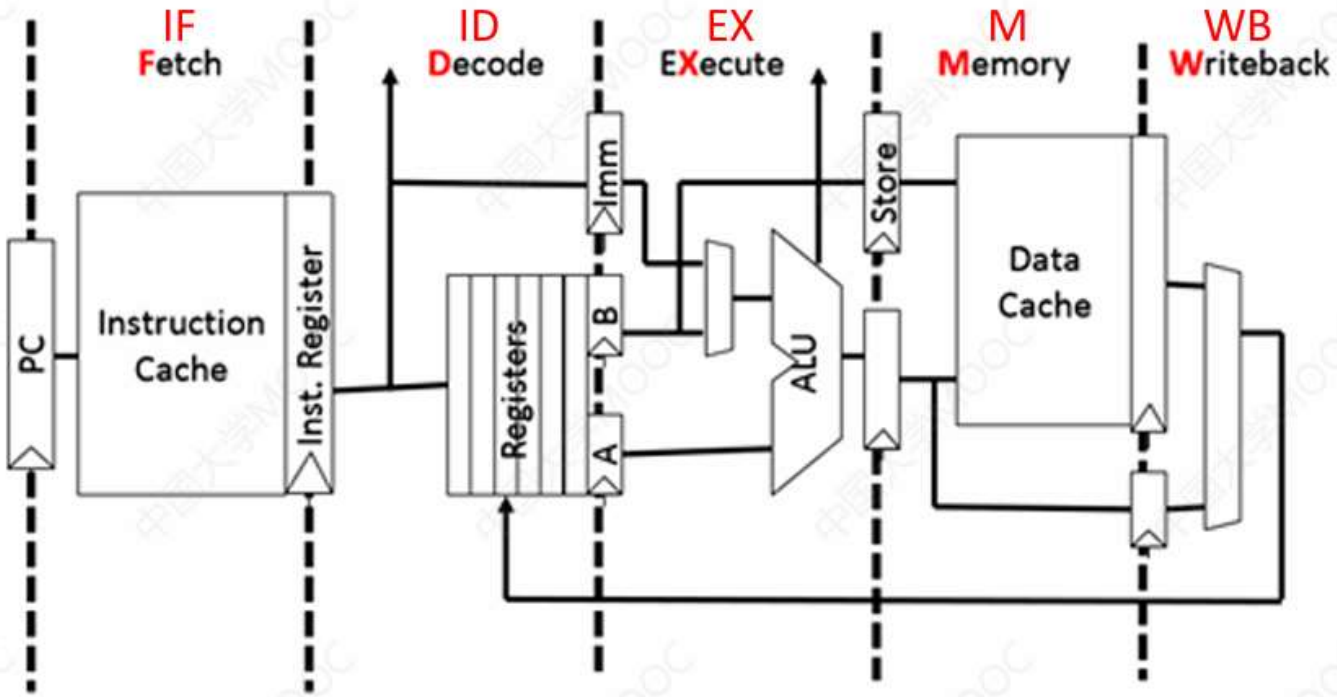
很多教材把写回PC的功能段称为“WrPC段”，其耗时比M段更短，可安排在M段时间内完成

①IF取指 → ②ID译码&取数 → ③EX 执行→ ④M访存 → ~~⑤WB写回寄存器~~

注：通常在IF段结束止之后PC就会自动 +“1”

| 指令的汇编格式 | 功能 |
| --- | --- |
| beq Rs, Rt, #偏移量 | 若(Rs)==(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC；否则(PC)+指令字长→PC |
| bne Rs, Rt, #偏移量 | 若(Rs)!=(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC；否则(PC)+指令字长→PC |

# 无条件转移指令的执行过程




转移类指令常采用相对寻址

无条件转移指令

IF：根据PC从指令Cache取指令至IF段的锁存器

ID：偏移量放入 Imm

EX：将目标PC值写回PC（左图没画全）

M：空段

WB：空段

①IF取指 → ②ID译码&取数 → ③EX 执行→ ~~④M访存~~ → ~~⑤WB写回寄存器~~

| 指令的汇编格式 | 功能 |
| --- | --- |
| jmp #偏移量 | (PC)+指令字长+(偏移量×指令字长)→PC |

“WrPC段”耗时比EX段更短，可安排在EX段时间内完成。WrPC段越早完成，就越能避免控制冲突。当然，也有的地方会在WB段时间内才修改PC的值

# 流水线冒险 1: 结构冒险 (资源冲突)

## 什么是结构冒险？

多条指令在同一时刻争用同一硬件资源所形成的冲突。

**最经典场景：** 访存冲突。如果数据和指令存放在同一个存储器中。Load指令在MEM阶段需要读取数据，而另一条指令在IF阶段需要读取指令，它们同时请求访问存储器，产生结构冒险。

## 解决办法

- **1. 暂停 (Stall / Bubble):** 让流水线在发生冲突的指令处暂停一个或几个时钟周期，等待资源释放。
- **2. 资源复制:** 设置独立的指令Cache和数据Cache（哈佛架构），这是现代CPU消除存储器结构冒险的标准做法。

# 流水线冒险 2: 数据冒险 (Data Hazards)

下一条指令需要用到前一条指令的计算结果，但前一条指令还没写回，导致操作数尚未准备好。

**写后读 (RAW)**

最常见的真数据相关

I1: ADD R1, R2, R3 (结果存R1, WB阶段写回)
I2: SUB R4, R1, R5 (用到R1, ID阶段读取) -> 此时R1还没被I1写回!

### 解决办法

1. **数据旁路 (Forwarding / Bypassing):** 不等写回阶段，直接把ALU的计算结果从EX段或MEM段引回作为下一条指令ALU的输入。（能解决大部分RAW）
2. **插入气泡 (Stall):** 如果是Load指令导致的RAW (Load-Use数据冒险)，数据只能在MEM段结束才拿到，旁路也来不及，必须暂停一拍流水线。
3. **编译优化:** 编译器通过调整指令顺序来避免相关。

# 流水线冒险 3: 控制冒险 (Control Hazards)

## 什么是控制冒险？

由于转移指令（如条件分支、无条件跳转、调用返回等）改变了PC的值，导致流水线预测取出的后续指令无效，必须清空流水线重新取指造成的性能损失。

在5段流水线中，通常要在EX段（甚至更晚）才能知道分支条件是否成立及目标地址，这会造成极大的性能浪费。

## 解决办法

- **1. 尽早判别转移:** 将分支转移的判断和目标地址的计算提前到ID阶段进行。
- **2. 分支预测 (Branch Prediction):**
  - 静态预测：简单假设总是转移或总是不转移。
  - 动态预测：利用分支历史表(BHT)记录历史行为，预测正确率可达90%+。
- **3. 延迟槽 (Delay Slot):** 在分支指令后插入一条不受该分支影响的有效指令。

# 高级流水线技术



## 超标量流水线 (Superscalar)

CPU内集成了**多条**指令流水线。在一个时钟周期内，可以并发发射多条指令（如多条算术指令和访存指令）。属于空间上的并行。不能调整指令的执行顺序，编译优化要求高。



## 超流水线 (Superpipeline)

通过增加流水线级数（切分得更细），进一步提高主频（缩短时钟周期）。在一个时钟周期内再分段。属于时间上的并行。流水线越长，发生冒险时的惩罚开销越大。



## 超长指令字 (VLIW)

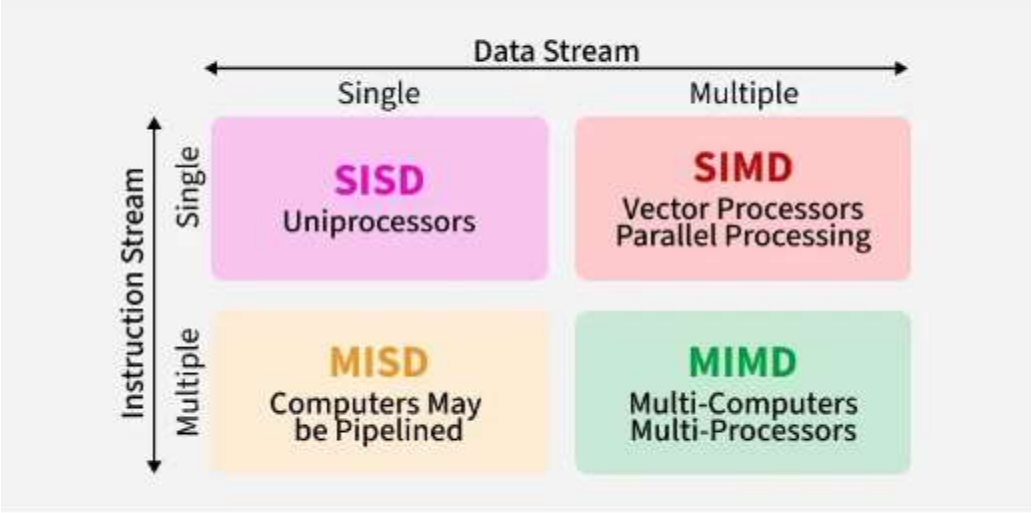
由**编译器**在编译时挖掘指令间的潜在并行性，将多条能并行执行的指令组合成一条具有多个操作码字段的“超长指令字”。硬件简单但依赖高端编译器。

* 此外，现代CPU还广泛使用**乱序执行 (Out-of-Order Execution)**，即动态流水线技术，利用保留站打乱指令执行顺序以避免阻塞。

# Flynn 分类法 (计算机体系结构分类)

按指令流和数据流的数量，Flynn将计算机系统分类：

- **SISD (单指令单数据流):** 传统的单核单体冯诺依曼机。每一时刻只有一个指令流和一个数据流被处理。
- **SIMD (单指令多数据流):** 经典如**向量处理器、GPU阵列**。一条指令同时对多个数据进行相同操作。极度适合图像和多媒体处理。
- **MISD (多指令单数据流):** 实际中不存在，仅为理论模型。
- **MIMD (多指令多数据流):** 现代的多核处理器、多处理器系统 (SMP, 集群)。每个处理器都有各自的控制单元，执行不同指令处理不同数据。



# 多核处理器与共享内存多处理器(SMP)

## 多核处理器 (Multi-core)

将两个或更多的独立处理核封装在一个芯片内（CMP技术）。

特点：

- 每个核有独立的L1 Cache，甚至L2 Cache。
- 各核共享最后一级Cache（如L3）或主存。
- 属于 Flynn 分类法中的 **MIMD** 架构。
- 通信机制极快，但受限于片上功耗和面积。

## 共享内存多处理器 (SMP)

Symmetric Multi-Processor，对称多处理器系统。

特点：

- 系统中有多个同构处理器，它们地位平等（对称）。
- 所有处理器**共享同一个物理主存空间**。
- 任何一个处理器访问主存任意单元的时间是相同的（UMA模型）。
- 面临极大的**缓存一致性**（Cache Coherence）问题。

## 硬件多线程

| | 细粒度多线程 | 粗粒度多线程 | 同时多线程（SMT） |
|---|---|---|---|
| 指令发射 | 轮流发射各线程的指令（每个时钟周期发射一个线程） | 连续几个时钟周期，都发射同一线程的指令序列，流水线阻塞时，切换另一个线程 | 一个时钟周期内，同时发射多个线程的指令 |
| 线程切换频率 | 每个时钟周期切换一次线程 | 只有流水线阻塞时才切换一次线程 | NULL |
| 线程切换代价 | 低 | 高，需要重载流水线 | NULL |
| 并行性 | 指令级并行，线程间不并行 | 指令级并行，线程间不并行 | 指令级并行，线程级并行 |

**26.** 【2016 统考真题】某计算机的主存储器空间为 4GB，字长为 32 位，按字节编址，采用 32 位字长指令字格式。若指令按字边界对齐存放，则程序计数器（PC）和指令寄存器（IR）的位数至少分别是（ ）。

A. 30, 30　　B. 30, 32　　C. 32, 30　　D. 32, 32

**26.** B

程序计数器（PC）用于指出下一条指令在内存中的地址，虽然可以用 32 位的地址来表示指令地址，但实际上内存中最多只能存放 4GB/32 位 $= 2^{30}$ 条指令，故可以用 30 位的字地址来表示指令地址，这种情况下指令必须采用按边界对齐的方式存放，故 PC 的位数至少是 30 位，即 PC 给出的地址是字地址。题干已说明指令按字边界对齐的方式存放，也就是说，指令地址都是 4 字节的整数倍，因此为了让 PC 的位数最少，可以采用字地址，取指令时将 PC 值左移 2 位到主存中取指令。指令寄存器（IR）用于存放从内存中取出的指令，它取决于指令字长，故 IR 的位数至少是 32 位。

17.【2011 统考真题】假定不采用 Cache 和指令预取技术，且机器处于“开中断”状态，则在下列有关指令执行的叙述中，错误的是（ ）。

A. 每个指令周期中 CPU 都至少访存一次

B. 每个指令周期一定大于或等于一个 CPU 时钟周期

C. 空操作指令的指令周期中任何寄存器的内容都不会被改变

D. 当前程序在每条指令执行结束时都可能被外部中断打断

17. C

由于不采用指令预取技术，每个指令周期都需要取指令，而不采用 Cache 技术，因此每次取指令都至少要访存一次（当指令字长与存储字长相等且按边界对齐时），A 正确。时钟周期是 CPU 的最小时间单位，每个指令周期一定大于或等于一个 CPU 时钟周期，B 正确。即使是空操作指令，在取指操作结束后，PC 也会自动加 1，C 错误。由于机器处于“开中断”状态，在每条指令执行结束时都可能被外部中断打断。

11.【2023 统考真题】数据通路由组合逻辑元件（操作元件）和时序逻辑元件（状态元件）组成。下列给出的元件中，属于操作元件的是（ ）。

I. 算术逻辑单元（ALU）　　II. 程序计数器（PC）

III. 通用寄存器组（GPRs）　　IV. 多路选择器（MUX）

A. 仅 I、II　　B. 仅 I、IV　　C. 仅 II、III　　D. 仅 I、II、IV

**11. B**

组合逻辑元件（操作元件）不含存储信号的记忆单元，任何时刻产生的输出仅取决于当前的输入，加法器、算术逻辑单元（ALU）、译码器、多路选择器、三态门等都属于操作元件。时序逻辑元件（状态元件）包含存储信号的记忆单元，各类寄存器和存储器，如通用寄存器组、程序计数器、状态/移位/暂存/锁存寄存器等，都属于状态元件。

**24.** 【2019 统考真题】下列有关处理器时钟脉冲信号的叙述中，错误的是（　）。

A. 时钟脉冲信号由机器脉冲源发出的脉冲信号经整形和分频后形成

B. 时钟脉冲信号的宽度称为时钟周期，时钟周期的倒数为机器主频

C. 时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定

D. 处理器总是在每来一个时钟脉冲信号时就开始执行一条新的指令

**24.** D

时钟脉冲信号的宽度称为时钟周期，时钟周期的倒数为机器主频。时钟脉冲信号由机器脉冲源发出的脉冲信号经整形和分频后形成，时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定。对于单周期 CPU，一个指令周期就是一个时钟周期，每个时钟周期执行一条新指令；对于多周期 CPU，每个指令周期（包含若干时钟周期）执行一条新指令；对于流水线 CPU，只有在理想情况下才能实现每个时钟周期执行一条新指令，选项 D 的描述有误。

**25.** 【2019 统考真题】某指令的功能为 R[r2]←R[r1]+M[R[r0]]，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（ ）。

I. 通用寄存器组（GPRs）　　II. 算术逻辑单元（ALU）

III. 存储器（Memory）　　IV. 指令译码器（ID）

A. 仅 I、II　　B. 仅 I、II、III　　C. 仅 II、III、IV　　D. 仅 I、III、IV

**25.** B

该指令的两个源操作数分别采用寄存器、寄存器间接寻址方式，因此在取数阶段需要用到通用寄存器组（GPRs）和存储器（Memory）；在执行阶段，两个源操作数相加需要用到算术逻辑单元（ALU）。而指令译码器（ID）用于对操作码字段进行译码，向控制器提供特定的操作信号，在取数及执行阶段用不到。

**26.** 【2021 统考真题】下列寄存器中，汇编语言程序员可见的是（ ）。

I. 指令寄存器　　II. 微指令寄存器

III. 基址寄存器　　IV. 标志/状态寄存器

A. 仅 I、II　　B. 仅 I、IV　　C. 仅 II、IV　　D. 仅 III、IV

**26.** D

汇编程序员可见的寄存器有基址寄存器（用于实现多道程序设计或者编制浮动程序）和状态/标志寄存器、程序计数器 PC 及通用寄存器组；而 MAR、MDR、IR 是 CPU 的内部工作寄存器，对汇编程序员不可见。微指令寄存器属于微程序控制器的组成部分，它是硬件设计者的任务，对汇编程序员是透明的（不可见的）。

**12.** 【2020 统考真题】下列关于“自陷”（Trap，也称陷阱）的叙述中，错误的是（　）。

A. 自陷是通过陷阱指令预先设定的一类外部中断事件

B. 自陷可用于实现程序调试时的断点设置和单步跟踪

C. 自陷发生后 CPU 将转去执行操作系统内核相应程序

D. 自陷处理完成后返回到陷阱指令的下一条指令执行

**12.** A

自陷是一种内部异常，A 错误。在 x86 中，用于程序调试的“断点设置”功能是通过自陷机制实现的，B 正确。执行到自陷指令时，无条件或有条件地自动调出操作系统内核程序进行执行，C 正确。CPU 执行陷阱指令后，会自动地根据不同陷阱类型进行相应的处理，然后返回到陷阱指令的下一条指令执行，D 正确。

**29.** 【2019 统考真题】在采用“取指、译码/取数、执行、访存、写回”5 段流水线的处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

I1: add s2, s1, s0　　//R[s2] ← R[s1] + R[s0]

I2: load s3, 0(t2)　　//R[s3] ← M[R[t2] + 0]

I3: add s2, s2, s3　　//R[s2] ← R[s2] + R[s3]

I4: store s2, 0(t2)　　//M[R[t2] + 0] ← R[s2]

下列指令对中，不存在数据冒险的是（　）。

A. I1 和 I3　　B. I2 和 I3　　C. I2 和 I4　　D. I3 和 I4

**29.** C

画出这四条指令在流水线中执行的过程如下图所示。

| 指令 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| add s2, s1, s0 | 取指 | 译码/取数 | 执行 | 访存 | 写回 | | | | | | | | | |
| load s3, 0(t2) | | 取指 | 译码/取数 | 执行 | 访存 | 写回 | | | | | | | | |
| add s2, s2, s3 | | | 取指 | | | | 译码/取数 | 执行 | 访存 | 写回 | | | | |
| store s2, 0(t2) | | | | | | | 取指 | | | | 译码/取数 | 执行 | 访存 | 写回 |

数据冒险即数据相关，指在程序中存在必须等前一条指令执行完才能执行后一条指令的情况，此时这两条指令即为数据相关。其中 I1 和 I3、I2 和 I3、I3 和 I4 均发生了写后读相关，因此必须等相关的前一条指令执行完才能执行后一条指令。只有 I2 和 I4 不存在数据冒险。

**30.** 【2020 统考真题】下列给出的处理器类型中，理想情况下，CPI 为 1 的是（ ）。

I. 单周期 CPU　II. 多周期 CPU　III. 基本流水线 CPU　IV. 超标量流水线 CPU

A. 仅 I、II　　B. 仅 I、III　　C. 仅 II、IV　　D. 仅 III、IV

**30.** B

CPI 表示执行指令所需的时钟周期数。对于一个程序或一台机器来说，其 CPI 是指执行该程序或机器指令集中的所有指令所需的平均时钟周期数。对于单周期 CPU，令指令周期 = 时钟周期，CPI = 1，I 正确。对于多周期 CPU，CPU 的执行过程分成几个阶段，每个阶段用一个时钟完成，每种指令所用的时钟数可以不同，CPI > 1，II 错误。对于基本流水线 CPU，让每个时钟周期流出一条指令，CPI = 1，III 正确。超标量流水线 CPU 在每个时钟周期内并发执行多条独立的指令，每个时钟周期流出多条指令，CPI < 1，IV 错误。

**31.** 【2023 统考真题】在采用“取指、译码/取数、执行、访存、写回”5 段流水线的 RISC 处理器中，执行如下指令序列（第一列为指令序号），其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1      add   s2, s1, s0        // R[s2]←R[s1] + R[s0]
I2      load  s3, 0(s2)         // R[s3]←M[R[s2] + 0]
I3      beq   t2, s3, L1        // if R[t2] = R[s3] jump to L1
I4      addi  t2, t2, 20        // R[t2]←R[t2] + 20
I5  L1: ……
```

若采用转发（旁路）技术处理数据冒险，采用硬件阻塞方式处理控制冒险，则在指令 I1～I4 的执行过程中，发生流水线阻塞的指令有（　）。

A. 仅 I3　　B. 仅 I2、I4　　C. 仅 I3、I4　　D. 仅 I2、I3、I4

**31**. C

I2 和 I1 之间存在数据冒险，I1 在 WB 段才将新值写回寄存器 R[s2]，但 I2 的 Ex 段就要读 R[s2]以计算访存的有效地址，I1 在 EX 段结束时就已生成 R[s2]的新值，被存放在 Ex/Mem 流水段寄存器中，采用转发技术后，可直接从该寄存器中取出数据送到 ALU 的输入端，这样 I2 执行时 ALU 用的是 R[s2]的新值，解决了 I2 和 I1 之间的数据冒险。I3 和 I2 之间存在数据冒险，属于 load-use 数据冒险，用转发电路无法解决 I3 和 I2 的数据相关问题，原因在于 I2 的功能是从内存中取数，只有在 Mem 段结束时才能从主存中得到 R[s3]的新值，但 I3 的 EX 段就要用到 R[s3]，因此无法用转发技术解决，I3 仍需阻塞一个时钟周期，等到 I2 的 Mem 段结束后，从 I2 的 Mem/Wb 流水段寄存器中取到 R[s3]的新值。I4 和 I3 之间存在控制冒险，beq 指令在 Ex 段设置条件码，在 Mem 段控制是否将转移地址送到 PC，这之后才能开始根据 PC 内容取指令，因此 I4 需要进行硬件阻塞。综上所述，I3 和 I4 的执行需要被阻塞，指令的执行过程如下表所示。

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I1 | IF | ID | EX | Mem | WB | | | | | | | |
| I2 | | IF | ID | EX | Mem | WB | | | | | | |
| I3 | | | IF | ID | 阻塞 | EX | Mem | WB | | | | |
| I4/I5 | | | | 阻塞 | 阻塞 | 阻塞 | 阻塞 | IF | ID | EX | Mem | WB |

**12.** 【2022 统考真题】下列关于并行处理技术的叙述中，不正确的是（ ）。

A. 多核处理器属于 MIMD 结构　　B. 向量处理器属于 SIMD 结构

C. 硬件多线程技术只可用于多核处理器

D. SMP 中所有处理器共享单一物理地址空间

**12.** C

MIMD 结构分为多计算机系统和多处理器系统。向量处理器是 SIMD 的变体，属于 SIMD 结构。硬件多线程技术在一个核中处理多个线程，可用于单核处理器，C 错误。共享内存多处理器（SMP）具有共享的单一物理地址空间，所有核都可通过存取指令来访问同一片主存地址空间。

某微程序控制器采用隐式（下址）与显式转移相结合的微指令格式，控制存储器（CM）容量为 $512 \times 36$位。微指令的操作控制字段采用字段直接编码法，分为 4 个互斥的微命令字段，分别包含 14、9、20、5 个互斥微命令（均已包含“无微操作”状态）。分支控制字段占 3 位，微指令中常数字段占 5 位（用于提供立即数或偏移量）。
若微地址码字段的位数由 CM 容量唯一确定，且在执行 INCR [R1]（主存单元自增：$M\left[[R1]\right] \leftarrow M\left[[R1]\right]+1$）指令的执行阶段（不含取指阶段）时，主存读/写均需 2 个微周期，ALU 运算需 1 个微周期，则该微指令中剩余的保留/扩展位数，以及该指令执行阶段最少需要的微指令条数分别是（ ）

A. 3 位，5 条
B. 3 位，6 条
C. 4 位，5 条
D. 4 位，6 条

【答案】：A

【解析】：微指令字段位数计算：

CM 容量为 $512 \times 36$位，微地址码字段长度为 $\log_2 512 = 9$位。

操作控制字段（字段直接编码法，需要向上取整 $\lceil \log_2 N \rceil$）：

字段 1：14 个状态 $\rightarrow \lceil \log_2 14 \rceil = 4$位

字段 2：9 个状态 $\rightarrow \lceil \log_2 9 \rceil = 4$位

字段 3：20 个状态 $\rightarrow \lceil \log_2 20 \rceil = 5$位

字段 4：5 个状态 $\rightarrow \lceil \log_2 5 \rceil = 3$位

微操作码字段总长度 $= 4 + 4 + 5 + 3 = 16$位。

已知：分支控制字段 3 位，常数字段 5 位，下址字段 9 位。

已用总位数 $= 16 + 3 + 5 + 9 = 33$位。

剩余保留位数 $= 36 - 33 = 3$位。

INCR [R1] 指令执行阶段微指令条数计算：

微周期 1：将 R1 中的地址送 MAR 并发出读命令：$\text{MAR} \leftarrow (\text{R1}), \text{Read}$（主存读第 1 周期）。

微周期 2：等待主存读完成，数据送 MDR：$\text{MDR} \leftarrow M[\text{MAR}]$（主存读第 2 周期）。

微周期 3：ALU 完成加 1 运算，结果送入 MDR：$\text{MDR} \leftarrow (\text{MDR}) + 1$（ALU 运算 1 周期）。

微周期 4：发出主存写命令：$\text{Write}, M[\text{MAR}] \leftarrow (\text{MDR})$（主存写第 1 周期）。

微周期 5：等待主存写完成（主存写第 2 周期）。

综上，执行阶段最少需要 5 个微周期，即 5 条微指令。

某 CPU 采用 5 级指令流水线（IF、ID、EX、MEM、WB），支持完全的数据旁路机制（Forwarding），寄存器采用前半周期写、后半周期读方式。分支指令（BEQ）在 EX 阶段完成条件判断与目标地址计算，分支预测策略为“始终不跳转（Not Taken）”。若预测错误，在 EX 阶段结束时冲刷 IF 和 ID 阶段的指令（分支惩罚为 2 个周期）。
数据 Cache 命中时 MEM 阶段耗时 1 个周期；发生 Cache 缺失时，在 MEM 阶段需额外插入 4 个 Stalls（即 MEM 阶段共需 5 个周期）。
现执行如下代码序列：

```
I1: LW   R1, 0(R2)      ; 从内存读取数据到 R1
I2: ADD  R3, R1, R4     ; R3 <- R1 + R4
I3: BEQ  R3, R5, Target ; 若 R3 == R5，跳转到 Target
I4: SUB  R6, R7, R8     ; 预测执行的顺序指令
```

假设指令 $I_1$发生数据 Cache 缺失；$I_2$ 依赖 $I_1$的结果 $R1$；$I_3$ 依赖 $I_2$的结果 $R3$，且实际执行时 $I_3$条件满足（即触发跳转）。从 $I_1$进入 IF 阶段（第 1 周期）开始，到目标指令 Target 进入 IF 阶段 为止，共经历的时钟周期数是（ ）

A. 10
B. 11
C. 12
D. 13

【答案】：B
【解析】：C1：$I_1$(IF)
C2：$I_1$(ID)，$I_2$(IF)
C3：$I_1$(EX)，$I_2$(ID)。在 ID 阶段检测到 $I_2$与 $I_1$存在 Load-Use 冲突，流水线本应控制 $I_2$停顿 1 周期。
C4：$I_1$ 进入 MEM 阶段，触发 Cache Miss（需在 MEM 停留 5 个周期，即 C4~C8）。$I_1$ 阻塞了 MEM 段，导致前面的流水段（EX、ID、IF）全线冻结/停顿。$I_2$ 继续留在 ID 阶段，$I_3$ 留在 IF 阶段。
C5 ~ C8：$I_1$ 持续在 MEM 阶段等待内存数据恢复。此时 Load-Use 所需的 1 周期停顿被 Cache Miss 的 5 周期阻塞完全掩盖（重叠）。
C9：$I_1$ 数据到达并进入 WB 阶段；$I_2$ 进入 EX 阶段（$R1$ 从 $I_1$的 MEM/WB 锁存器旁路转发至 $I_2$的 EX 段）；$I_3$ 进入 ID 阶段；$I_4$ 进入 IF 阶段。
C10：$I_2$ 进入 MEM 阶段（运算结果 $R3$产生）；$I_3$ 进入 EX 阶段（$R3$ 从 $I_2$的 EX/MEM 锁存器直接旁路转发至 $I_3$的 EX 段，无需额外停顿！）。在 C10 结束时，$I_3$ 在 EX 阶段确认跳转成立（预测错误），流水线向 IF 和 ID 段发出冲刷信号（清空处于 ID 的 $I_4$和处于 IF 的 $I_5$）。
C11：分支目标指令 Target 被正确装入，进入 IF 阶段。
综上所述，到 Target 进入 IF 阶段共经历 11 个时钟周期。

关于单核心指令流水线中的中断与异常处理机制，下列说法中正确的是（ ）。
Ⅰ. 在 5 级流水线中，若第 $i$条指令在 EX 段触发“算术溢出异常”，而第 $i-1$条指令在 MEM 段触发“访存缺页异常”，CPU 将优先响应第 $i-1$条指令的缺页异常
Ⅱ. 缺页异常（Page Fault）属于“故障（Fault）”，CPU 在执行完缺页异常处理程序后，返回的目标地址是触发该异常指令的下一条指令地址
Ⅲ. 外部可屏蔽中断（INTR）在指令执行周期的末尾（或流水线指令写回/提交阶段）进行采样；若响应中断，关中断并保存的断点是下一条即将执行的指令地址
Ⅳ. 自陷（Trap，如系统调用指令 syscall）属于预期的软件中断，CPU 响应 Trap 异常后，保存的断点为当前自陷指令的地址

A. 仅 Ⅰ、Ⅲ
B. 仅 Ⅰ、Ⅳ
C. 仅 Ⅱ、Ⅲ
D. 仅 Ⅰ、Ⅱ、Ⅳ

【答案】：A

【解析】：Ⅰ 正确：流水线采用精确异常机制，异常的响应与提交严格按照程序中的指令顺序（Program Order）进行。由于第 $i-1$ 条指令在程序顺序上早于第 $i$ 条指令，尽管第 $i$ 条指令更早被检测出溢出，CPU 也会暂存第 $i$ 条指令的异常，优先响应第 $i-1$ 条指令的缺页异常，并清空流水线中第 $i$ 条及后续指令。

Ⅱ 错误：缺页异常属于“故障（Fault）”。故障是可修复的异常，软件修复（如调页入内存）后，CPU 需要重新执行引发故障的指令，因此保存的断点是当前触发异常指令本身的地址，而非下一条指令地址。

Ⅲ 正确：外部中断（INTR）是异步事件，在指令边界（如流水线写回/提交阶段）统一采样。响应中断后，当前指令已经顺利完成，因此保存的断点是下一条尚未执行的指令地址。

Ⅳ 错误：自陷（Trap）是预期的编程引发事件（如 syscall）。处理完自陷服务程序后，程序应当继续向下执行，因此保存的断点是自陷指令的下一条指令地址（若保存当前地址会导致死循环）。

**某单总线 CPU 的数据通路结构中，ALU 包含一个暂存器 Y（连接 ALU 的一个输入端）和输出暂存器 Z（连接 ALU 的输出端），通用寄存器为 R0~R7，内存读写通过 MAR 和 MDR 完成（MDR 包含内部总线接口 $MDR_{in}/MDR_{out}$ 和外部总线接口 $MDR_{inE}/MDR_{outE}$）。主存读/写操作均需 1 个时钟周期。**

**假设指令 ADD [R1], R2（功能：$M[[R1]] \leftarrow M[[R1]] + (R2)$）已被取至指令寄存器 IR 中。若不考虑取指阶段，则执行该指令需要的最少时钟周期数，以及倒数第二个时钟周期内激活的控制信号组合分别是（ ）**

**A. 5 个周期，** $Z_{out}, MDR_{in}$
**B. 6 个周期，** $Z_{out}, MDR_{in}$
**C. 6 个周期，** $R2_{out}, ADD, Z_{in}$
**D. 7 个周期，** $Z_{out}, MAR_{in}$

【答案】：B
【解析】：不考虑取指阶段，分析 ADD [R1], R2 在单总线上的执行过程：
C1：将 R1 中的主存地址送 MAR
微操作：MAR ← (R1)
控制信号：$R1_{out}, MAR_{in}$
C2：读主存，将操作数读入 MDR
微操作：MDR ← $M$[MAR]
控制信号：$Read, MDR_{inE}$
C3：将 MDR 中的主存操作数送入 ALU 暂存器 Y
微操作：$Y \leftarrow$ (MDR)
控制信号：$MDR_{out}, Y_{in}$
C4：将 R2 的值与 Y 相加，结果存入暂存器 Z
微操作：$Z \leftarrow (R2) + (Y)$
控制信号：$R2_{out}, ADD, Z_{in}$
C5（倒数第二个周期）：将 Z 中计算好的加法结果送回 MDR，准备写主存
微操作：MDR ← $(Z)$
控制信号：$Z_{out}, MDR_{in}$
C6（最后一个周期）：启动写主存操作
微操作：$M$[MAR] ← (MDR)
控制信号：$Write, MDR_{outE}$
总结：最少需要 6 个时钟周期；倒数第二个周期（即 C5）激活的控制信号组合为 $Z_{out}, MDR_{in}$。

某双发射（Dual-issue）超标量处理器采用顺序发射、乱序执行的流水线架构，包含 1 个整数 ALU（执行耗时 1 周期）和 1 个 Load/Store 单元（执行耗时 2 周期）。现有一段汇编代码序列如下：

代码段

```
I1: LW   R1, 0(R2)   ; R1 <- M[R2]
I2: ADD  R3, R1, R4  ; R3 <- R1 + R4
I3: SUB  R5, R6, R7  ; R5 <- R6 - R7
I4: ADD  R1, R8, R9  ; R1 <- R8 + R9
```

关于该指令序列在流水线中的数据相关性与执行控制逻辑，下列说法中正确的是（ ）

A. $I_1$与 $I_2$存在 RAW（写后读）相关，可以通过寄存器重命名（Register Renaming）技术消除该相关性

B. 在顺序发射机制下，若 $I_2$因等待 $I_1$的结果而无法发射，无数据相关的 $I_3$可以越过 $I_2$提前与 $I_1$在同一周期发射

C. $I_2$与 $I_4$存在 WAR（读后写）相关，若不采用寄存器重命名，流水线必须保证 $I_4$不能在 $I_2$读取 R1 之前更新 R1

D. $I_1$与 $I_4$存在 WAR 相关，$I_2$ 与 $I_4$存在 WAW 相关

【答案】：C

【解析】：数据相关性剖析：

$I_1$写 R1，$I_2$ 读 R1 →RAW（写后读，真相关）。

$I_1$写 R1，$I_4$ 写 R1 →WAW（写后写，名相关/反相关）。

$I_2$读 R1，$I_4$ 写 R1 →WAR（读后写，名相关/反相关）。

选项判定：

A 错误：RAW 是真数据相关（涉及真实的数据传递），必须等待前指令产生结果，无法通过寄存器重命名消除。寄存器重命名仅能消除由寄存器名字冲突引发的“伪相关”（WAR 和 WAW）。

B 错误：该处理器为顺序发射（In-order Issue），必须严格按照程序顺序（$I_1, I_2, I_3, I_4$）检视与发射指令。若 $I_2$无法发射，发射窗口被阻塞，后续的 $I_3$绝不能跨越 $I_2$提前发射。

C 正确：$I_2$ 依赖 $I_1$产生的 R1 旧值，而 $I_4$将对 R1 写入新值（WAR 相关）。在乱序执行但没有寄存器重命名的情况下，如果 $I_4$执行过快并在 $I_2$读取 R1 之前将新值写入了物理寄存器 R1，会导致 $I_2$读到错误的新值。因此必须硬件加锁保障 WAR 时序。

D 错误：颠倒了相关性类型（$I_1$ 与 $I_4$是 WAW，$I_2$ 与 $I_4$是 WAR）。